In [3]:
```
#importing of libraries
import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt
```

In [4]:
```
#Load Dataset and perform EDA (Exploratory Data Analysis)
df = pd.read_csv(r'C:\Users\HP\Downloads\c107fa55-45be-4f9c-8f61-088e88d1fb0c.csv')
```

In [5]:
```
#View five first rows
df.head()
```

Out[5]:

	TV	Radio	Social Media	Influencer	Sales
0	Low	3.518070	2.293790	Micro	55.261284
1	Low	7.756876	2.572287	Mega	67.574904
2	High	20.348988	1.227180	Micro	272.250108
3	Medium	20.108487	2.728374	Mega	195.102176
4	High	31.653200	7.776978	Nano	273.960377

In [6]:
```
#View the columns
df.columns
```

Out[6]:
```
Index(['TV', 'Radio', 'Social Media', 'Influencer', 'Sales'], dtype='str')
```

In [7]:
```
#rows and columns number
df.shape
```

Out[7]:
```
(572, 5)
```

In [8]:
```
#Missing values per column
df.isnull().sum()
```

Out[8]:
```
TV              0
Radio           0
Social Media    0
Influencer      0
Sales           0
dtype: int64
```

In [9]:
```
#dtypes and non-null columns
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 572 entries, 0 to 571
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   TV            572 non-null    str
 1   Radio         572 non-null    float64
 2   Social Media  572 non-null    float64
 3   Influencer    572 non-null    str
 4   Sales         572 non-null    float64
dtypes: float64(3), str(2)
memory usage: 22.5 KB
```

In [10]:
```
df.describe()
```

Out[10]:

	Radio	Social Media	Sales
count	572.000000	572.000000	572.000000
mean	17.520616	3.333803	189.296908
std	9.290933	2.238378	89.871581
min	0.109106	0.000031	33.509810
25%	10.699556	1.585549	118.718722
50%	17.149517	3.150111	184.005362
75%	24.606396	4.730408	264.500118
max	42.271579	11.403625	357.788195

In [ ]:

In [11]:
```
#convertining the strings to int values
df = pd.get_dummies(df, columns=['TV', 'Influencer'],
    drop_first=True,dtype=int)
```

In [12]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 572 entries, 0 to 571
Data columns (total 8 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   Radio             572 non-null    float64
 1   Social Media      572 non-null    float64
 2   Sales             572 non-null    float64
 3   TV_Low            572 non-null    int64  
 4   TV_Medium         572 non-null    int64  
 5   Influencer_Mega   572 non-null    int64  
 6   Influencer_Micro  572 non-null    int64  
 7   Influencer_Nano   572 non-null    int64  
dtypes: float64(3), int64(5)
memory usage: 35.9 KB
```

In [13]:
```
print(df)
```

```
        Radio  Social Media       Sales  TV_Low  TV_Medium  Influencer_Mega  \
0    3.518070      2.293790   55.261284       1          0                0   
1    7.756876      2.572287   67.574904       1          0                1   
2   20.348988      1.227180  272.250108       0          0                0   
3   20.108487      2.728374  195.102176       0          1                1   
4   31.653200      7.776978  273.960377       0          0                0   
..        ...           ...         ...     ...        ...              ...   
567 14.656633      3.817980  191.521266       0          1                0   
568 28.110171      7.358169  297.626731       0          0                1   
569 11.401084      5.818697  145.416851       0          1                0   
570 21.119991      5.703028  209.326830       0          1                0   
571 13.221237      3.660566  135.773151       1          0                0   

     Influencer_Micro  Influencer_Nano  
0                   1                0  
1                   0                0  
2                   1                0  
3                   0                0  
4                   0                1  
..                ...              ...  
567                 1                0  
568                 0                0  
569                 0                1  
570                 0                0  
571                 1                0  

[572 rows x 8 columns]
```

In [14]:
```
#Check for multicollinearity among independent variables (X) using correlation matrices and VIF
#Correlation Matrix
corr_matrix = df[['TV_Low', 'TV_Medium', 'Influencer_Mega', 'Radio', 'Social Media', 'Sales', 'Influencer_Micro
                  'Influencer_Nano']].corr()

print(corr_matrix)
```

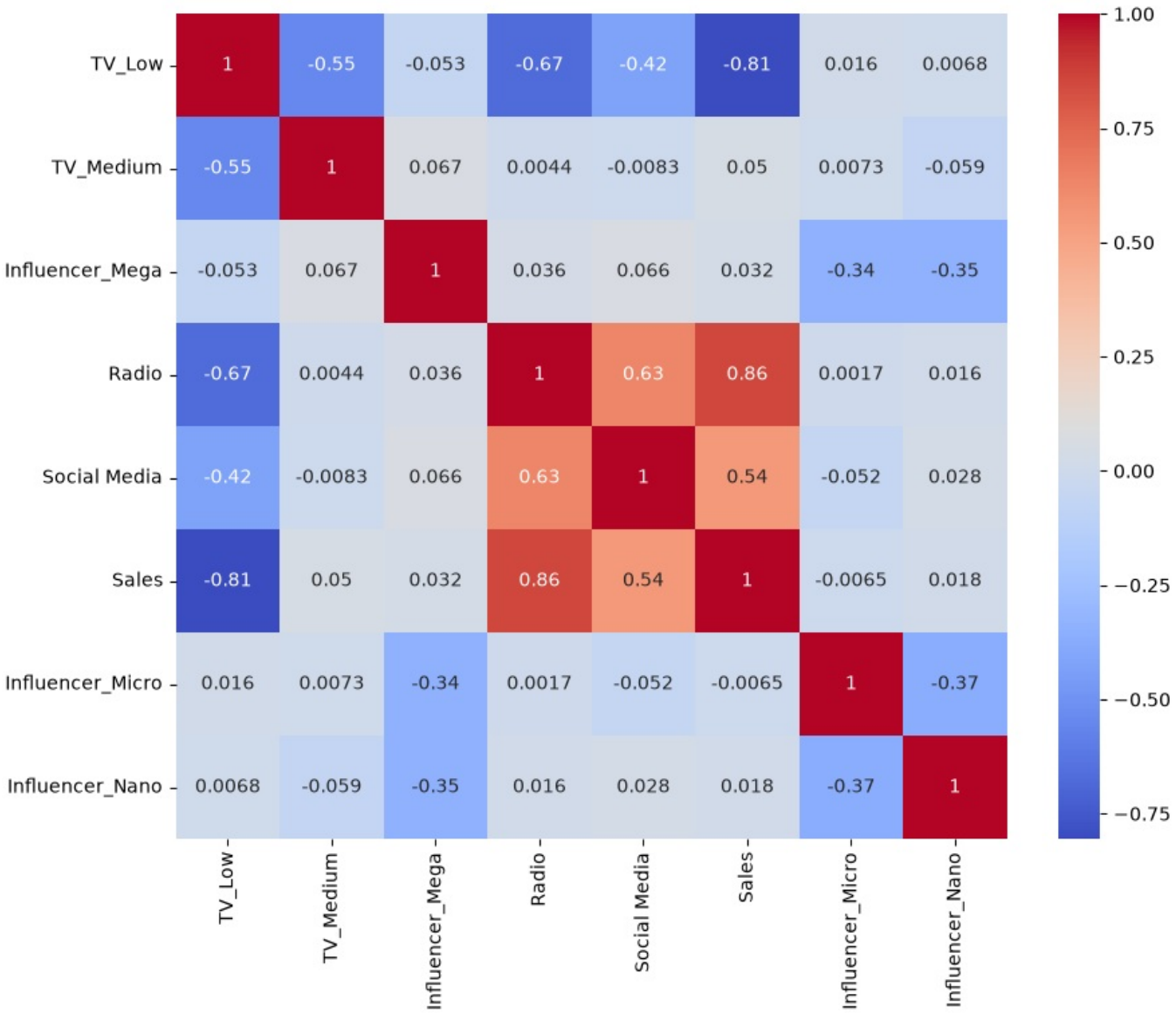
```
                  TV_Low  TV_Medium  Influencer_Mega     Radio  \
TV_Low          1.000000  -0.550117        -0.052697 -0.674203   
TV_Medium      -0.550117   1.000000         0.067488  0.004400   
Influencer_Mega -0.052697   0.067488         1.000000  0.035704   
Radio          -0.674203   0.004400         0.035704  1.000000   
Social Media   -0.423910  -0.008272         0.066023  0.629941   
Sales          -0.805896   0.050449         0.032443  0.858036   
Influencer_Micro  0.016107   0.007302        -0.336096  0.001712   
Influencer_Nano   0.006815  -0.059374        -0.345177  0.015640   

                  Social Media     Sales  Influencer_Micro  Influencer_Nano  
TV_Low               -0.423910 -0.805896          0.016107         0.006815  
TV_Medium            -0.008272  0.050449          0.007302        -0.059374  
Influencer_Mega       0.066023  0.032443         -0.336096        -0.345177  
Radio                 0.629941  0.858036          0.001712         0.015640  
Social Media          1.000000  0.542048         -0.052077         0.028030  
Sales                 0.542048  1.000000         -0.006503         0.017656  
Influencer_Micro     -0.052077 -0.006503          1.000000        -0.368361  
Influencer_Nano       0.028030  0.017656         -0.368361         1.000000  
```

In [15]:
```
#Visualization of corr_matrix
plt.figure(figsize=(10,8))

sns.heatmap(corr_matrix,annot=True,cmap='coolwarm')
```

```
plt.show()
```



In [16]:
```
#VIF (Variance Inflation Factor)
from statsmodels.stats.outliers_influence import variance_inflation_factor
```

In [17]:
```
X = df[['TV_Low', 'TV_Medium', 'Influencer_Mega', 'Radio', 'Social Media', 'Influencer_Micro',
                'Influencer_Nano']]
```

In [18]:
```
X = df.drop('Sales', axis=1)

Y = df['Sales']
```

In [ ]:
```
print(X.head())
```

In [ ]:
```
print(Y.head())
```

In [22]:
```
# Calculate VIF for each column
#Interpretation
#VIF    Meaning
#1–5    Acceptable
#5–10   Moderate concern
#>10    Serious multicollinearity
vif_data = pd.DataFrame()
vif_data['Feature'] = X.columns
vif_data['VIF'] = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]

print(vif_data.sort_values('VIF', ascending=False))
```

```
            Feature        VIF
0             const  31.649565
3            TV_Low   4.076384
1             Radio   3.479641
4         TV_Medium   2.233350
2      Social Media   1.669098
7   Influencer_Nano   1.627430
6  Influencer_Micro   1.618434
5   Influencer_Mega   1.593889
```

In [20]:
```
#Building Multiple Linear Regression Model
import statsmodels.api as sm
X = sm.add_constant(X)
```

In [21]:
```
# 1. Make sure X is numeric + has constant
X = X.astype(float)
X = sm.add_constant(X)
```

In [23]:
```
model = sm.OLS(Y, X).fit()
```

In [24]:
```
#Evaluate Model Performance
print(model.summary())
```

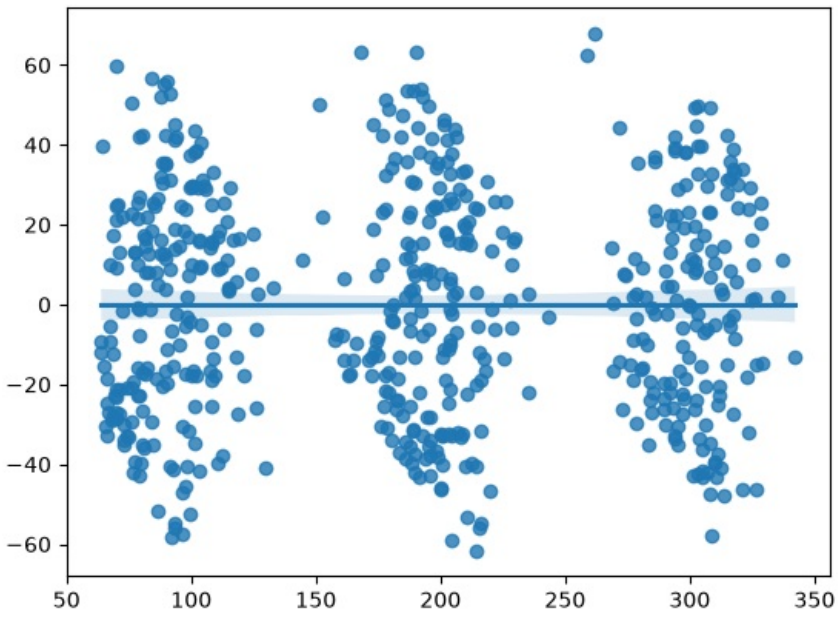
```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  Sales   R-squared:                       0.904
Model:                            OLS   Adj. R-squared:                  0.903
Method:                 Least Squares   F-statistic:                     760.4
Date:                Mon, 22 Jun 2026   Prob (F-statistic):          1.82e-282
Time:                        15:12:21   Log-Likelihood:                -2713.4
No. Observations:                 572   AIC:                             5443.
Df Residuals:                     564   BIC:                             5478.
Df Model:                           7
Covariance Type:            nonrobust
================================================================================
                   coef    std err          t      P>|t|      [0.025      0.975]
--------------------------------------------------------------------------------
const          217.4784      6.584     33.031      0.000     204.546     230.411
Radio            2.9735      0.235     12.644      0.000       2.512       3.435
Social Media    -0.1391      0.676     -0.206      0.837      -1.467       1.189
TV_Low        -154.5736      4.949    -31.231      0.000    -164.295    -144.852
TV_Medium      -75.5947      3.647    -20.726      0.000     -82.759     -68.431
Influencer_Mega    2.4948      3.462      0.721      0.471      -4.305       9.295
Influencer_Micro   2.9391      3.378      0.870      0.385      -3.695       9.574
Influencer_Nano    0.8015      3.346      0.240      0.811      -5.770       7.373
==============================================================================
Omnibus:                       58.711   Durbin-Watson:                   1.874
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               17.802
Skew:                           0.057   Prob(JB):                     0.000136
Kurtosis:                       2.143   Cond. No.                         147.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

In [ ]:
```
#Approximately 90.4% of the variation in sales is explained by advertising expenditure and influencer category
#The Adjusted R² value of 0.903 indicates that approximately 90.3% of the variability in sales is explained by
#The regression model as a whole is statistically significant.
#The overall regression model was statistically significant (F = 760.4, p < 0.001), indicating that at least on
```

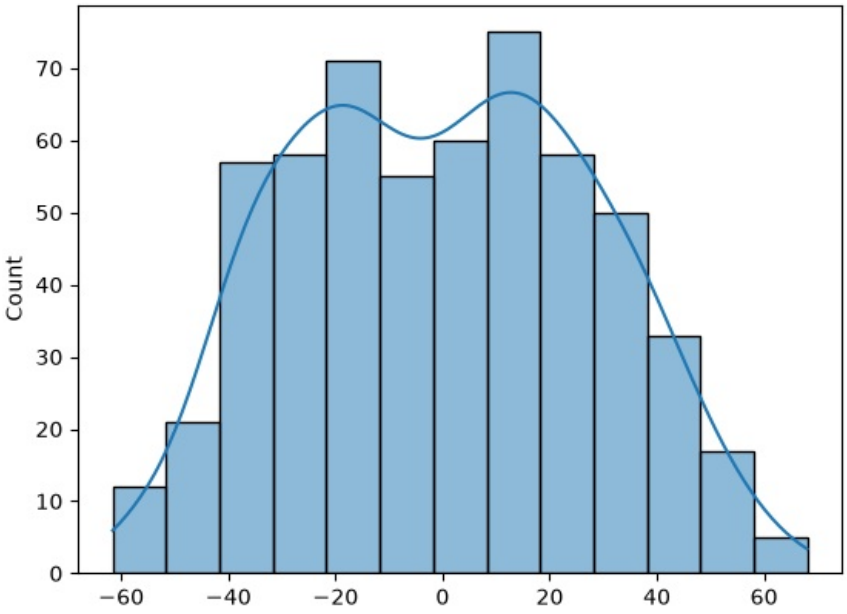
In [28]:
```
#Diagnostic Plots
#Linearity
sns.regplot(x=model.fittedvalues,
            y=model.resid)
```

Out[28]: <Axes: >



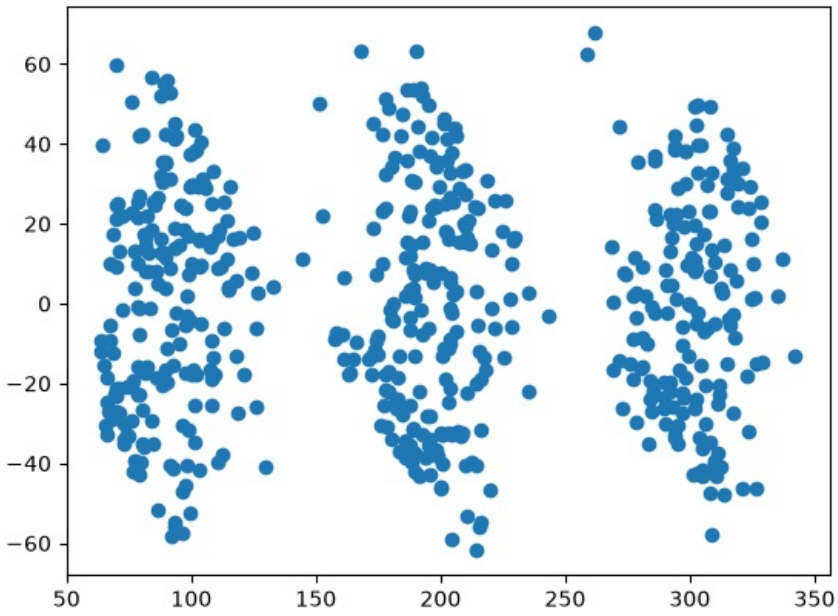
In [27]:
```
#Normality
sns.histplot(model.resid, kde=True)
```

Out[27]: <Axes: ylabel='Count'>



In [26]:
```
#Homoscedasticity
plt.scatter(model.fittedvalues,
            model.resid)
```

Out[26]: <matplotlib.collections.PathCollection at 0x1bbea96a660>



In [ ]:
```
#Business Recommendation using the coef and p-value
#The regression model explained approximately 90.3% of the variation in sales and was statistically significant
```